

UE02 : Exercices - Fonctions d'ordre supérieur et lambdas

Module : M323 · **Compétences ICT** : 1.4, 2.1 · **Langage** : C# (.NET 8)

Bandes travaillées : C1B et C2B · **Fil rouge** : catalogue de véhicules

Le corrigé se trouve dans un fichier séparé. Les exercices 2 à 5 utilisent le même projet console.

Où travailler

Activité	Lieu	Fichier
Diagnostic et exercice 1	Sur cette fiche	Aucun code à saisir
Exercices 2 à 5	Visual Studio, application console .NET 8	Program.cs
Extension	Visual Studio, même projet	Program.cs

Préparer le projet Visual Studio

1. Créez une **Application console** en C# avec **.NET 8**.
2. Nommez la solution `M323_UE02_VotreNom`.
3. Ajoutez les trois fichiers suivants : `Voiture.cs`, `Donnees.cs`, `Program.cs`.
4. Copiez le kit de départ placé à la fin de cette fiche dans les fichiers correspondants.
5. Lancez avec `Ctrl+F5`. Le programme doit afficher `12 voitures chargées`.

Chaque exercice indique précisément les fichiers nécessaires. Aucun opérateur LINQ, notamment `where` ou `Select`, n'est autorisé dans cette UE.

Diagnostic - Ranger ou appeler

SUR CETTE FICHE · **Durée** : 5 min · **Antériorité** : rappel UE01 · **Non noté**

Observez :

```
Func<Voiture, string> etiquette = EtiquetteDe;  
string texte = etiquette(catalogue[0]);  
Console.WriteLine(texte);
```

1. Entourez la ligne qui range une fonction.
2. Soulignez les lignes qui appellent une fonction.
3. Dans `Func<Voiture, string>`, écrivez le type d'entrée et le type de sortie.

Si les trois réponses sont justes, passez immédiatement à l'exercice 1.

Exercice 1 - Reconnaître un critère

SUR CETTE FICHE · Niveau : Guidé · Durée : 8 min · Antériorité : rappel UE01 puis nouveau ·
Compétence ICT : 1.4 · Non noté

Pour chaque signature, choisissez `Func`, `Action` ou `Predicate`. Écrivez le type complet.

```
static int Age(Voiture v) { /* ... */ }  
static void Afficher(Voiture v) { /* ... */ }  
static bool EstHybride(Voiture v) { /* ... */ }  
static string Resumer(Voiture v, bool avecPrix) { /* ... */ }  
static void Terminer() { /* ... */ }
```

Questions :

1. Pour `EstHybride`, `Func<Voiture, bool>` serait-il possible ?
2. Pourquoi `Predicate<Voiture>` est-il plus lisible dans ce cas ?
3. Quelle information de la signature permet de choisir sans lire le corps ?

Cette activité vérifie le vocabulaire ICT 1.4. Elle ne valide pas C1B à elle seule.

Exercice 2 - Injecter un critère

À CODER DANS VISUAL STUDIO · Fichier : `Program.cs` · Dépendances : `Voiture.cs`, `Donnees.cs`
Niveau : Guidé · Durée : 20 min · Antériorité : nouveau · Compétence ICT : 2.1 · Matrice : C2B

1. Écrivez cette méthode et complétez son corps :

```
static void Afficher(  
    List<Voiture> catalogue,  
    Predicate<Voiture> critere)  
{  
    // à compléter  
}
```

2. Écrivez trois méthodes nommées : `EstElectrique` , `EstDiesel` , `EstRecente` .

3. Appelez `Afficher` trois fois en passant les méthodes sans parenthèses.

4. Écrivez `Compter(List<Voiture>, Predicate<Voiture>)` , qui renvoie le nombre de voitures acceptées.

5. Utilisez `Compter` pour obtenir le nombre de voitures de plus de 100 000 km.

6. Vérifiez : les nombres attendus sont 3 électriques, 3 diesels, 5 voitures depuis 2021 et 2 voitures de plus de 100 000 km.

Répondez ensuite en quatre lignes : donnée reçue, comportement reçu, moment de l'appel du comportement, résultat produit par `Compter` .

Exercice 3 - Remplacer les méthodes par des lambdas

À CODER DANS VISUAL STUDIO · Fichier : `Program.cs` · Dépendances : exercice 2

Niveau : Semi-guidé · Durée : 20 min · Antériorité : nouveau · Compétence ICT : 1.4 · Matrice : C2B

1. Reprenez les trois appels à `Afficher` de l'exercice 2.

2. Remplacez chaque méthode de critère par une lambda à corps court.

3. Le corps d' `Afficher` ne doit pas changer.

4. Écrivez une lambda à corps long pour la règle suivante : une voiture est une bonne affaire si son prix divisé par son âge plus un est inférieur à 3 000. Utilisez 2026.

5. Parmi les écritures suivantes, indiquez celle qui ne compile pas et corrigez-la :

```
Predicate<Voiture> a = v => v.Prix > 20000;  
Predicate<Voiture> b = v => { v.Prix > 20000; };  
Predicate<Voiture> c = v => { return v.Prix > 20000; };
```

6. Écrivez deux situations dans lesquelles vous conserveriez une méthode nommée.

Exercice 4 - Fabriquer un critère et observer la capture

À CODER DANS VISUAL STUDIO · **Fichier** : Program.cs · **Dépendances** : méthode Afficher de l'exercice 2

Niveau : Semi-guidé · **Durée** : 20 min · **Antériorité** : nouveau · **Compétence ICT** : 1.4, 2.1 · **Matrice** : C2B

1. Écrivez une fabrique :

```
static Predicate<Voiture> PlusCherQue(int seuil)
{
    // renvoyer un critère, ne tester aucune voiture ici
}
```

2. Fabriquez deux critères avec les seuils 20 000 et 35 000.

3. Utilisez-les avec Afficher .

4. Expliquez pourquoi PlusCherQue(20000) ne renvoie pas un booléen.

5. Prédisez l'affichage avant d'exécuter :

```
int seuil = 20000;
Predicate<Voiture> critere = v => v.Prix > seuil;
seuil = 40000;
Console.WriteLine(Compter(catalogue, critere));
```

6. Vérifiez votre prédiction. Nommez la variable capturée et expliquez ce que la closure conserve réellement.

7. Modifiez le code pour que le critère reste lié à 20 000 sans dépendre d'une variable modifiée ensuite.

Exercice 5 - Consolidateur : callbacks et explication d'algorithme

À CODER DANS VISUAL STUDIO · Fichier : Program.cs · Dépendances : exercices 2 à 4

Niveau : Autonome · Durée : 25 min · Antériorité : nouveau · Compétences ICT : 1.4, 2.1 · Matrice : C1B, C2B

Vous devez écrire une méthode d'inventaire configurable :

```
static void Inventorier(  
    List<Voiture> catalogue,  
    Predicate<Voiture> critere,  
    Action<Voiture> quandTrouve,  
    Action quandTermine)  
{  
    // à compléter  
}
```

Intention de l'exercice

Cet exercice vérifie que vous séparez :

- le **mécanisme stable** (parcourir la liste, décider quand appeler) ;
- les **comportements variables** (filtrer, agir par voiture, agir à la fin).

En d'autres termes : `Inventorier` orchestre, les callbacks décrivent la règle métier.

1. Appelez `Inventorier` pour les voitures de plus de 100 000 km.
2. Pour chaque voiture trouvée, affichez son modèle et son prix.
3. À la fin, affichez le nombre de voitures et la somme des prix.
4. `Inventorier` ne doit contenir ni la règle des 100 000 km, ni le format d'affichage, ni les variables de total.
5. Vérifiez que `quandTermine` est appelé une seule fois, même si aucune voiture ne correspond.

Geste à maîtriser (ICT + matrice)

- **ICT 1.4** : choisir et écrire correctement `Predicate<Voiture>` , `Action<Voiture>` , `Action` .
- **ICT 2.1** : étendre le comportement sans modifier le mécanisme de parcours.
- **C2B** : passer les callbacks et les appeler au bon endroit.
- **C1B** : expliquer le rythme d'appel (souvent/parfois/une fois) avec précision.

Modèle déclaratif attendu (exemple)

```
int total = 0;
int somme = 0;

Inventorier(
    catalogue,
    v => v.Kilometrage > 100000,
    v =>
    {
        Console.WriteLine($"{v.Modele} : {v.Prix} CHF");
        total++;
        somme += v.Prix;
    },
    () => Console.WriteLine($"{total} voitures, {somme} CHF"));
```

Vérification demandée : pour une nouvelle demande métier (par exemple "électrique"), vous changez les callbacks, pas la boucle de `Inventorier`.

Preuve C1B

Expliquez l'algorithme en six lignes maximum :

- la donnée reçue ;
- les trois comportements reçus ;
- le moment exact où chacun est appelé ;
- ce que la méthode sait ;
- ce qu'elle ignore ;
- ce qui changerait pour inventorier une autre catégorie.

Le code seul valide C2B. L'explication précise de l'algorithme valide C1B.

Extension - Préparer les paliers I et A

OPTIONNEL, POUR ALLER PLUS LOIN · À CODER DANS VISUAL STUDIO · Durée indicative :
20 min

Cette extension n'est pas nécessaire pour terminer le parcours commun. Elle prépare la combinaison de comportements qui sera évaluée plus tard.

Écrivez :

```
static Predicate<Voiture> Et(  
    Predicate<Voiture> premier,  
    Predicate<Voiture> second)  
{  
    // à compléter  
}
```

Puis :

1. combinez « électrique » et « prix inférieur à 35 000 » ;
2. combinez « récente » et « kilométrage inférieur à 50 000 » ;
3. vérifiez les résultats sur le catalogue ;
4. inversez l'ordre des deux critères ;
5. expliquez quand l'ordre pourrait changer le coût du traitement, même si le résultat reste identique ;
6. proposez une méthode ou sans recopier le parcours du catalogue.

Auto-positionnement

Ancrage : B = 3,0 ; I = 4,5 ; A = 6,0. L'UE02 construit le palier B nécessaire pour atteindre I dans les UE04 et UE05.

Bande C1 - Mettre en oeuvre la programmation fonctionnelle

- ☐ C1B : j'explique l'algorithme d'une fonction d'ordre supérieur en nommant ses données, ses comportements et leur moment d'appel.
- ☐ C1I : sera travaillé en UE04 par le découpage et la composition d'un pipeline.
- ☐ C1A : sera travaillé en UE07 par l'analyse de l'exactitude et de la lisibilité.

Bande C2 - Fonctions d'ordre supérieur

- ☐ C2B : je range une fonction, je la passe en argument, je la renvoie et j'applique un comportement à une liste.
- ☐ C2I : sera travaillé en UE04 par la combinaison de map, filter et reduce.
- ☐ C2A : sera travaillé en UE05 par un traitement de données complexe.

Check-list ICT de l'UE02

- ☐ 1.4 Je choisis entre `Func`, `Action` et `Predicate` à partir d'une signature.
- ☐ 1.4 J'écris une lambda à corps court et une lambda à corps long.
- ☐ 1.4 J'explique ce qu'une closure capture.
- ☐ 1.4 Je définis un callback et j'indique qui choisit le moment de son appel.
- ☐ 2.1 J'écris une méthode qui reçoit un comportement.
- ☐ 2.1 J'écris une méthode qui renvoie un comportement.
- ☐ 2.1 J'explique l'algorithme sans me limiter à réciter les types.

Kit de départ

Voiture.cs

```
public record Voiture(  
    string Marque,  
    string Modele,  
    int Prix,  
    string Carburant,  
    int Kilometrage,  
    int Annee);
```

Donnees.cs

```
public static class Donnees
{
    public static List<Voiture> ChargerCatalogue()
    {
        return new List<Voiture>
        {
            new("Renault", "Zoe", 22000, "électrique", 15000, 2021),
            new("VW", "Golf", 18000, "diesel", 120000, 2016),
            new("Tesla", "Model 3", 41000, "électrique", 30000, 2022),
            new("Peugeot", "208", 14500, "essence", 68000, 2018),
            new("Toyota", "Yaris", 19500, "hybride", 42000, 2020),
            new("BMW", "Serie 1", 27000, "diesel", 95000, 2017),
            new("Fiat", "Panda", 8900, "essence", 130000, 2012),
            new("Skoda", "Octavia", 23500, "diesel", 88000, 2019),
            new("Hyundai", "Kona", 31000, "électrique", 12000, 2023),
            new("Audi", "A3", 34000, "essence", 45000, 2021),
            new("Dacia", "Sandero", 12000, "essence", 76000, 2019),
            new("Volvo", "XC40", 52000, "hybride", 9000, 2024),
        };
    }
}
```

Program.cs

```
List<Voiture> catalogue = Donnees.ChargerCatalogue();
Console.WriteLine(catalogue.Count + " voitures chargées.");

// Écrivez les appels ici.

// Écrivez les méthodes sous cette ligne.
```